

宁波大学信息学院

实验报告

实验课程名称： 嵌入式系统

学　　院： 信息科学与工程学院

专　　业： 计算机科学与技术

指 导 教 师： 王翀,王艺睿

报告人姓名： 顾凯琪 **学号：** 226001624

班　　级： 2022 级计算机 2 班

学　　期： 2024-2025 学年第二学期

信息学院实验中心制

实验								
成绩								

实验项目名称	实验四——ARM 中断实验

一、实验目的与要求：

示例将利用 Exynos4412 的 K2、K3 这 2 个 I/O 引脚的中断模式，当被按下时进入相应的中断处理函数处理相应的事件。

二、实验设备：

1. FS-JTAG 仿真器、USB 转串口线、电源
2. PC 机（OS：Windows 7）
3. Eclipse for ARM
4. GCC 编译工具
5. Yagarto 工具包
6. JRE
7. Exynos4412 实验箱

三、实验方法（原理、流程图）

原理：

1、 电路原理

电路原理如图所示，K2、K3 分别与 GPX1_1、GPX1_2 相连，在没有按下按键时 GPX1_1、GPX1_2 引脚上一直处于高电平，当把这两个引脚设为中断模式并为下降沿中断，则按键被按下俩引脚就会有高电平变为低电平，因此，产生 GPIO 中断进入相应的中断函数，处理中段事件，从终端上打印出相应的按键信息。其中 K2 对应的是 XEINT9 中断源，K3 对应的是 XEINT10 中断源。

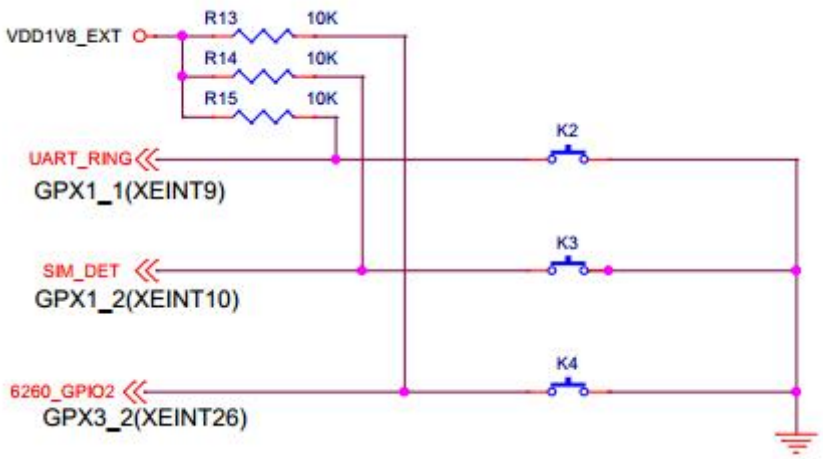


图 EXYNOS4412 中断实验电路图

2、编程流程

（1）设置 GPX1_2、GPX1_2 两个管脚没有内部上下拉属性，然后配置为中断模式；

（2）设置中断触发方式；

（3）GPIO 控制器中关闭屏蔽、使能中断；

（4）在 GIC 中断控制器中使能中断；

（5）设置中断优先级；

（6）使能 GIC；

（7）选择中断发送给 CPU0；

（8）等待中断产生，然后进入中断处理器函数；

（9）清楚中断源的挂起状态。

流程图：

流程：

|---> 1. 导入汇编工程

| |---> 打开 Eclipse 开发工具，添加 Ket_Int 工程

|---> 2. 编译程序

| |---> 点击编译按钮进行程序编译

|---> 3. 调试工程

|---> a. 配置 FS-JTAG 调试工具

| |---> 打开 FS-JTAG 软件，连接目标板

|---> b. 配置调试工具

| |---> 配置 Main 选项

| |---> 配置 Debugger 选项

| |---> 配置 Command 选项

|---> c. 进入调试

|---> 点击调试按钮进行调试

四、实验过程、步骤及内容

说明：部分实验图标注日期为 2014 年，为实验指导书中清晰示例图，仅为丰富实验指导书内容所置，不影响实验报告撰写。

实验内容：

1、 寄存器设置

为了实现进入中断目的，需要通过配置 GPX1CON 寄存器将 GPX1_1、GPX1_2 设置为中断模式。设置中断方式、中断处理函数、使能中断。

2、 程序编写

相关代码如下：

（1） 设置异常向量表

注意:C12 寄存器用于设定向量表的基地址,这个地址和 map.lds 中的连接地址一致,是 0x40008000。读者理解后，可以更改测试。

```
    .text
.global _start
_start:
        b       reset
        ldr     pc,_undefined_instruction
        ldr     pc,_software_interrupt
        ldr     pc,_prefetch_abort
        ldr     pc,_data_abort
        ldr     pc,_not_used
        ldr     pc,_irq
        ldr     pc,_fiq

```

```
_undefined_instruction: .word  _undefined_instruction
_software_interrupt:    .word  _software_interrupt
_prefetch_abort:        .word  _prefetch_abort
_data_abort:            .word  _data_abort
_not_used:              .word  _not_used
_irq:                   .word  irq_handler
_fiq:                   .word  _fiq

reset:

    ldr r0,=0x40008000
    mcr p15,0,r0,c12,c0,0       @ Vector Base Address Register
    ……
```

（2） 编写中断处理函数

汇编中的处理

```
        /****  irq_handler  ****/
irq_handler:

    sub  lr,lr,#4
    stmfd sp!,{r0-r12,lr}
    bl  do_irq   //跳转到 do_irq 处理器函数
    ldmfd sp!,{r0-r12,pc}^
```

主程序中的中断初始化

```
   /*-------------------------MAIN FUNCTION-----------------------------*/
/**************************************************************************
 * @brief       Main program body
 * @param[in]   None
 * @return      int
 *************************************************************************/
int main(void)
{
    //LED2 GPX2_7
    GPX2.GPX2CON |= 0x1 << 28;
    //LED3 GPX1_0
    GPX1.GPX1CON |= 0x1;
    //Led4 GPF3_4
    GPF3.GPF3CON |= 0x1 << 16;

    //Key_2  Interrupt  GPX1_1
    GPX1.GPX1PUD = GPX1.GPX1PUD & ~(0x3 << 2); // Disables Pull-up/Pull-down
    GPX1.GPX1CON = (GPX1.GPX1CON & ~(0xF << 4)) | (0xF << 4); //GPX1_1: WAKEUP_IN
```

```
T1[1](EXT_INT41[1])
    EXT_INT41_CON = (EXT_INT41_CON & ~(0x7 << 4)) | 0x2 << 4;
    EXT_INT41_MASK = (EXT_INT41_MASK & ~(0x1 << 1)); //  Bit: 1 = Enables interru
pt

    //Key_3  Interrupt  GPX1_2
    GPX1.GPX1PUD = GPX1.GPX1PUD & ~(0x3 << 4); // Disables Pull-up/Pull-down
    GPX1.GPX1CON = (GPX1.GPX1CON & ~(0xF << 8)) | (0xF << 8); //GPX1_2:WAKEUP_INT
1[2] (EXT_INT41[2])
    EXT_INT41_CON = (EXT_INT41_CON & ~(0x7 << 8)) | 0x2 << 8;
    EXT_INT41_MASK = (EXT_INT41_MASK & ~(0x1 << 2)); //  Bit: 1 = Enables interru
pt


 7
    /*
     * GIC interrupt controller:
     * */

    // Enables the corresponding interrupt SPI25, SPI26 -- Key_2, Key_3
    ICDISER.ICDISER1 |= (0x1 << 25) | (0x1 << 26);

    CPU0.ICCICR |= 0x1; //Global enable for signaling of interrupts

    CPU0.ICCPMR = 0xFF; //The priority mask level.Priority filter. threshold

    ICDDCR = 1;     //Bit1:  GIC monitors the peripheral interrupt signals and
                    //      forwards pending interrupts to the CPU interfaces2

    ICDIPTR.ICDIPTR14 = 0x01010101; //SPI25  SPI26  interrupts are sent to proces
sor 0

    printf("\n ********* GIC test ********\n");

    while (1){
        GPF3.GPF3DAT |= 0x1 << 4;
        mydelay_ms(500);
        GPF3.GPF3DAT &= ~(0x1 << 4);
        mydelay_ms(500);
    }

    return 0;
}
```

中断处理函数

```
1   /*********************************************************************
```

```
 * @brief       IRQ Interrupt Service Routine program body
 * @param[in]   None
 * @return      None
 *****************************************************************/
void do_irq(void )
{
    int irq_num;
    irq_num = (CPU0.ICCIAR & 0x1FF);
    switch (irq_num) {

    case 58: //turn on LED2; turn off LED3
        GPX2.GPX2DAT = 0x1 << 7;
        GPX1.GPX1DAT &= ~0x1;
        printf("IRQ interrupt !! turn on LED2; turn off LED3\n");

        //Clear Pend
        EXT_INT41_PEND |= 0x1 << 2;
        ICDICPR.ICDICPR1 |= 0x1 << 26;
        break;
    case 57: //Turn on Led3; Turn off Led2

        GPX2.GPX2DAT &= ~(0x1 << 7);
        GPX1.GPX1DAT |= 0x1;
        printf("IRQ interrupt !! Turn on LED3; Turn off LED2\n");

        //Clear Pend
        EXT_INT41_PEND |= 0x1 << 1;
        ICDICPR.ICDICPR1 |= 0x1 << 26;
        break;
    }

    // End of interrupt
    CPU0.ICCEOIR = (CPU0.ICCEOIR & ~(0x1FF)) | irq_num;
}
```

代码解释：

主函数，主要功能是初始化外设并开启中断：

首先，LED2、LED3、LED4 和 Key_2、Key_3 引脚的 GPIO 配置被设置为输出或输入。然后，外部中断控制器（GIC）被配置，包括使能 SPI25、SPI26 中断，以及设置中断的优先级和处理器。在主循环中，LED4（GPF3_4）会每隔 500ms 闪烁一次。主函数最后返回 0。

中断处理函数：

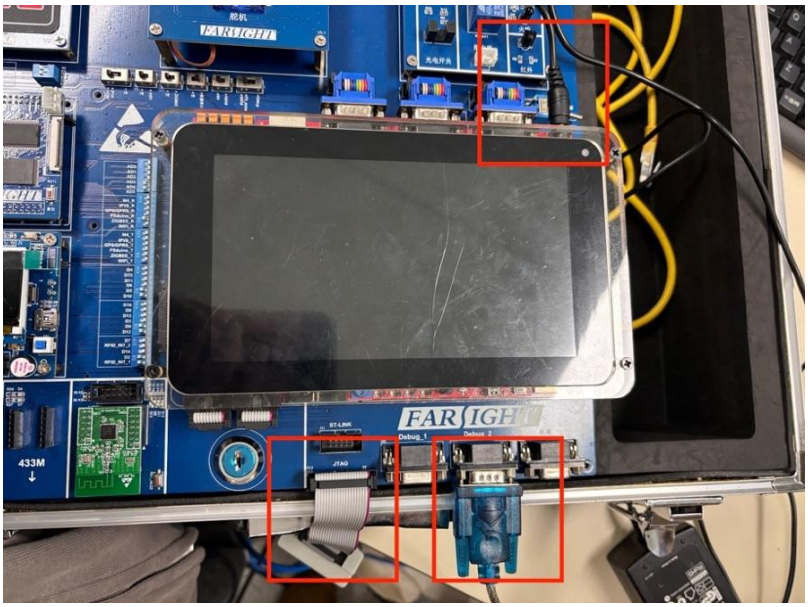
`void do_irq(void )`该函数在每次中断事件发生时被调用，在这段代码中，有两种中断号，分别是 57 和 58。当中断号为 57 时，函数会关闭 LED2 并打开 LED3；当中断号为 58 时，

函数会关闭 LED3 并打开 LED2。在每个情况下，函数都会打印一条消息，说明哪个中断事件被触发了。最后，函数会清除中断挂起标志，并将该中断事件的中断优先级降为最低。最后，函数结束时，将处理完的中断号发送给中断控制器以通知其该中断已经被处理完毕。

实验步骤：

1. 连接硬件平台

按下图所示，连接仿真器、USB 转串口线、电源。



后续步骤（由于实验一实验报告已详细说明，此处不加以赘述）：

1) USB 转串口驱动安装
2) Putty 串口终端配置
3) 关闭开发板电源，将拨码开关 SW1 调至 0110(EMMC 启动模式)，然后打开电源。

2. 导入工程源码

参考第 1 章节的 ARM 开发环境搭建部分。

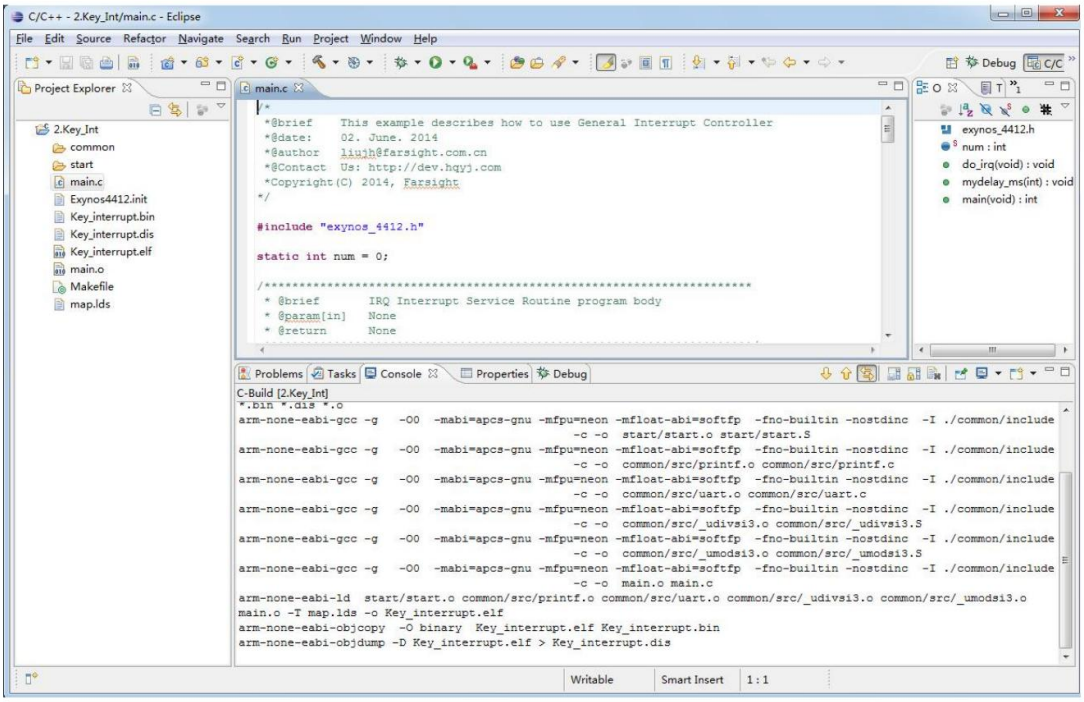
光盘实验源码路径：华清远见-CORTEXA9 资料\实验代码\1. ARM 体系结构与接口技术部分\实验代码\05-Ket_Int

要把它放到其他地方去，比如 D 盘中（路径中不可以有中文）。

编译程序

工程导入成功后，点击编译图标（或者快捷键“Ctrl + B”）

编译成功后如下图所示：



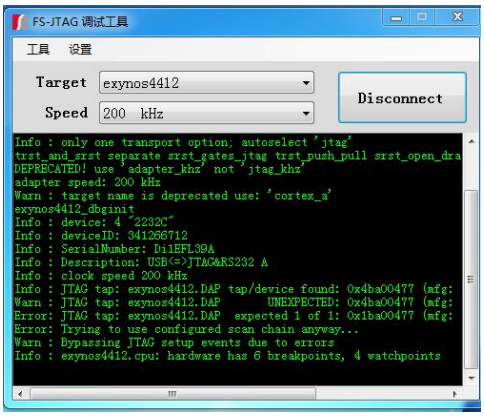
3. 调试工程

a. 配置 FS-JTAG 调试工具

打开 FS-JTAG 软件，在 Target 选项中选择 exynos4412，通信速率设置 200kHz。单击 Connect 按钮后，该按钮会变为 Disconnect，如图所示，即表示已经连接目标板。由于当前开发板还没有运行程序，仿真器无法获取相关信息，所以显示错误报告，是正常现象。

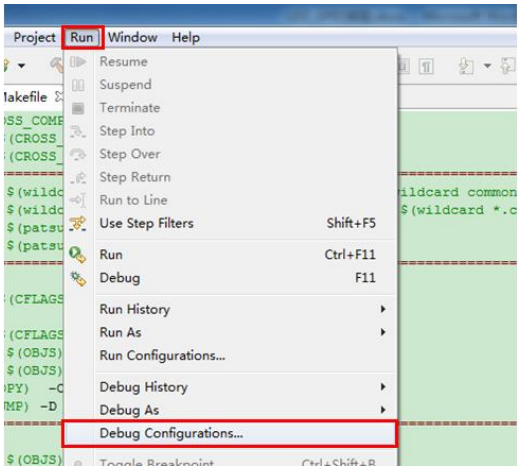
通信速率可以选择 200KHz，如果出现连接不稳定的情况，则将速率调制 100KHz。

注意：该软件在 win7 以上的 windows 操作系统中请使用管理员模式打开，否则会出现软件崩溃的现象。

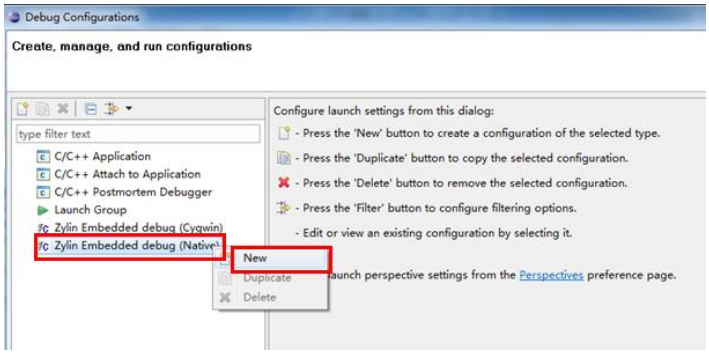


b. 配置调试工具

选中“Run”→“Debug Configuras…”

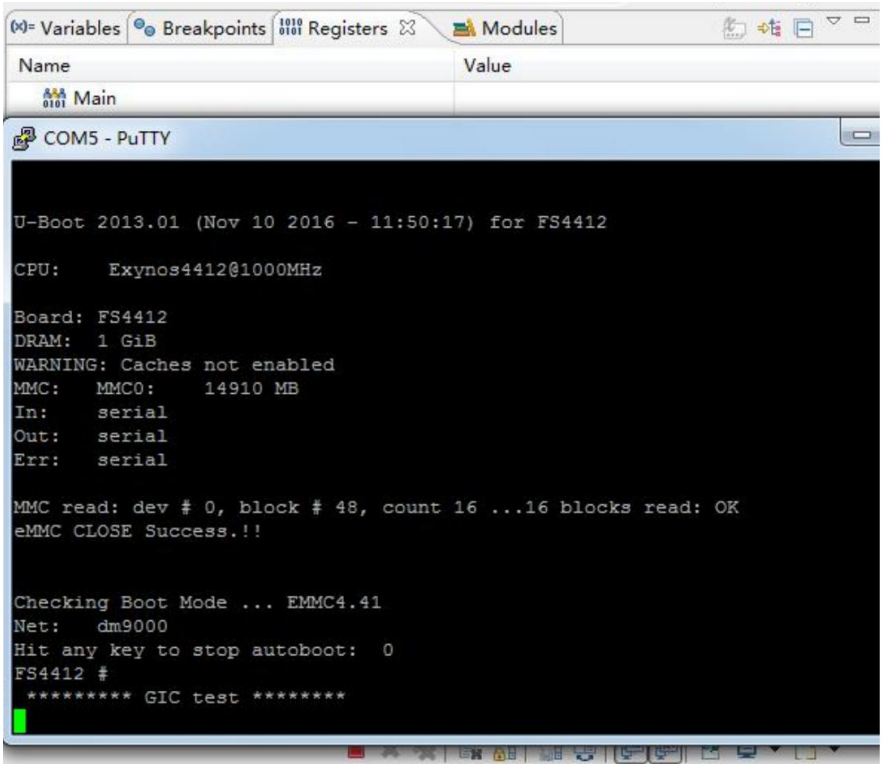


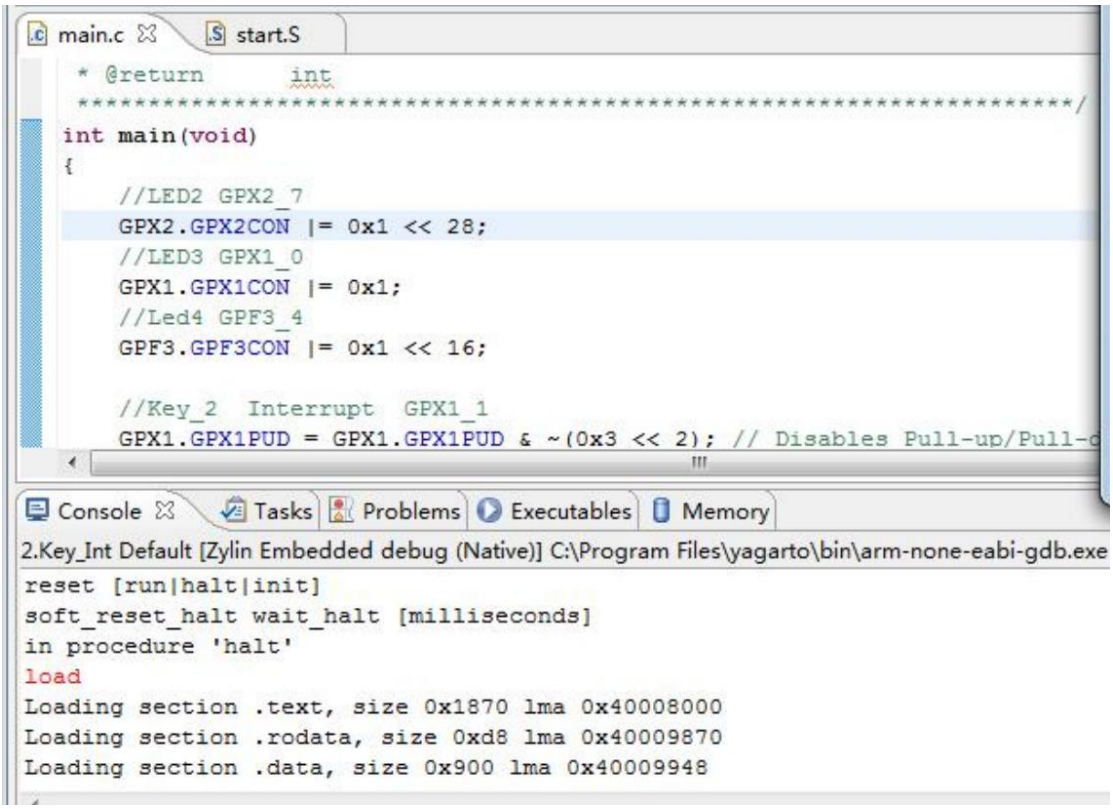
出现如下所示窗口，在“Zylin Embedded debug (Native)”选项上右击鼠标，在点击“New”选项



点击“New”

【串口终端配置】





【main 选项卡】

然后再点击“Main”选项栏中红色框内的“Browse”选项选择将要执行的程序，调试的程序格式为 xx.**elf** 的文件。找到工程对应的目录，选中目录中的 **ket_int.elf** 文件，然后选择“打开”。

【Debugger 选项卡】

在 Debugger 选项卡中的 Main 子选项卡中的 GDB debugger 的框中单击“Browse”按钮选择前面安装的 arm-none-eabi-gdb.exe（这里选择自己安装 gcc 编译工具的安装目录 C:\Program Files\yagarto\bin），在 GDB Command file 中选择自己工程目录下的 Exynos4412.init 文件。

注：在 Debugger 选项卡中有一个 Stop on startup at：选项，如调试工程为 c 工程，有 main 函数则勾选该选项、否则不勾选。

【Command 选项卡】

在 Command 选项中，如下图所示，在“Initialize’ commands”中添加 3 行命令

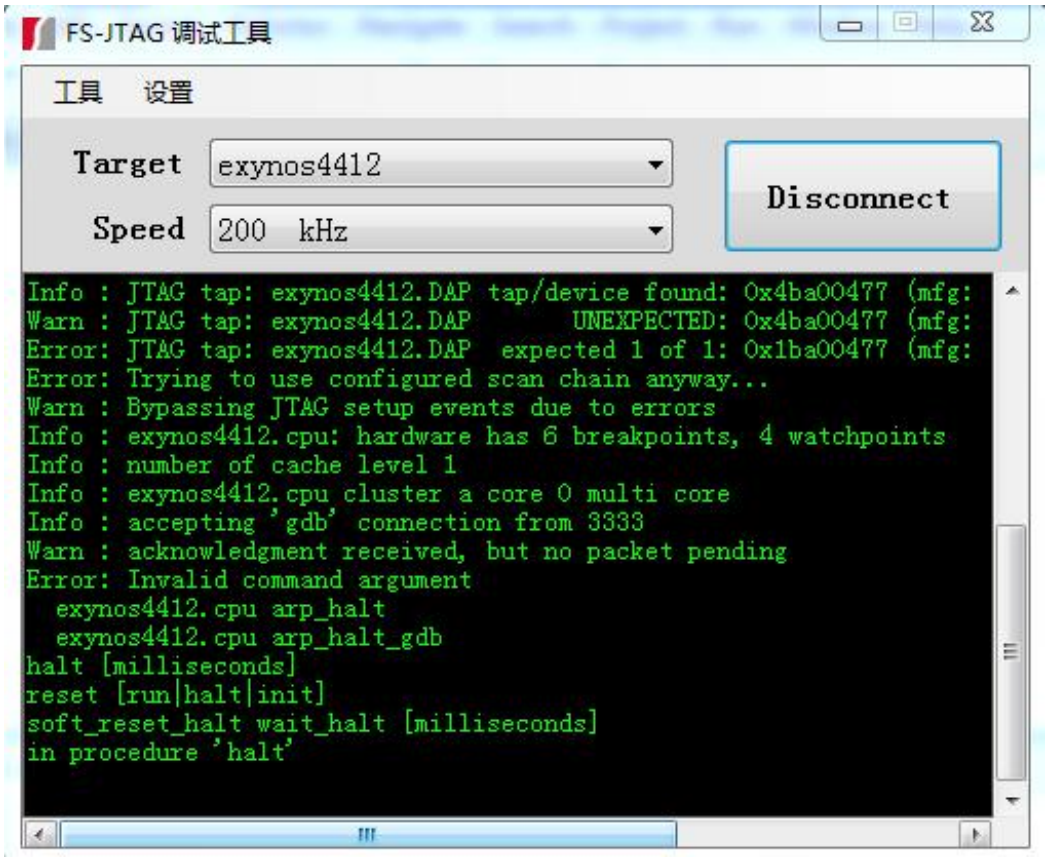
```
load
break main
c
```

添加完后点击如下图所示的“Apply”，调试选项配置完成。再点击“Debug”选项进入就可以进行调试了。

c. 进入调试

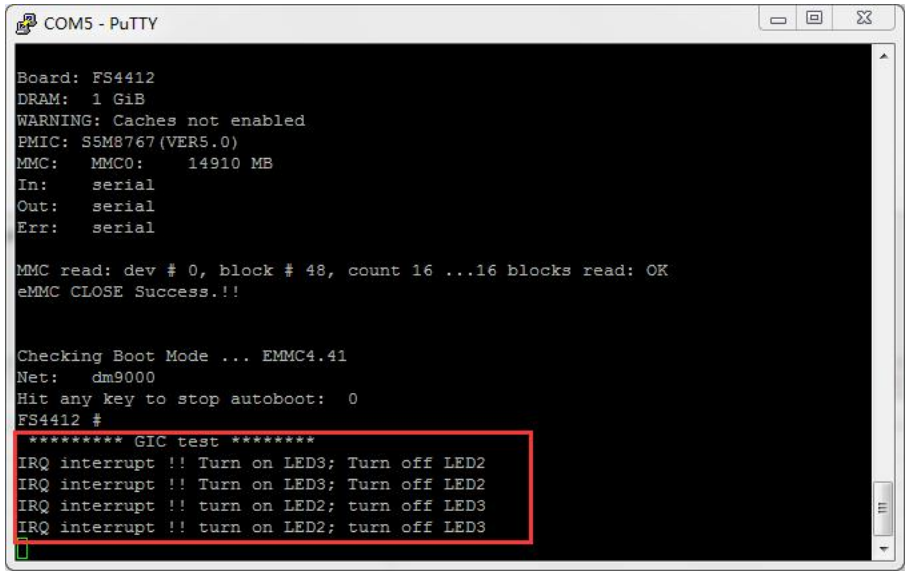
进入调试界面后，点击全速运行  的调试按钮。

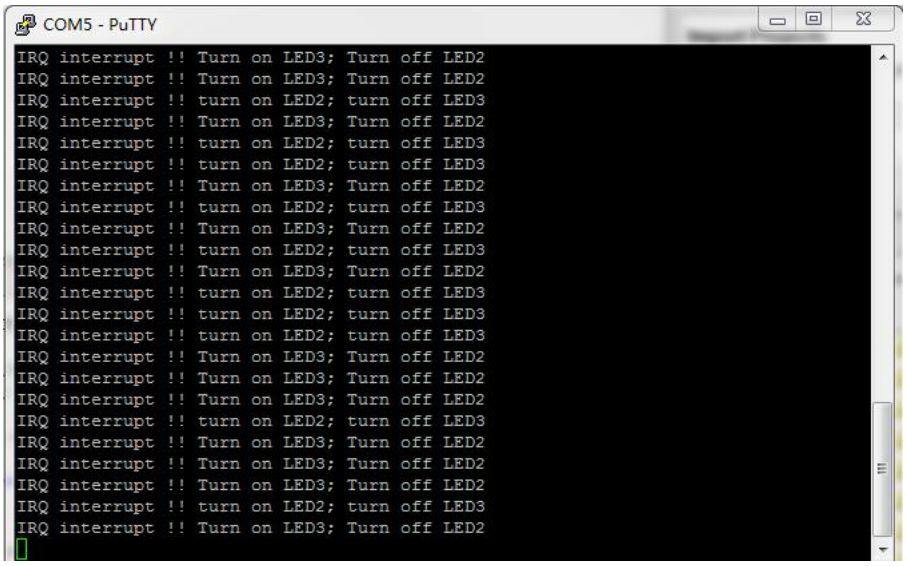
此时表示结束：



五、实验数据（现象）处理分析

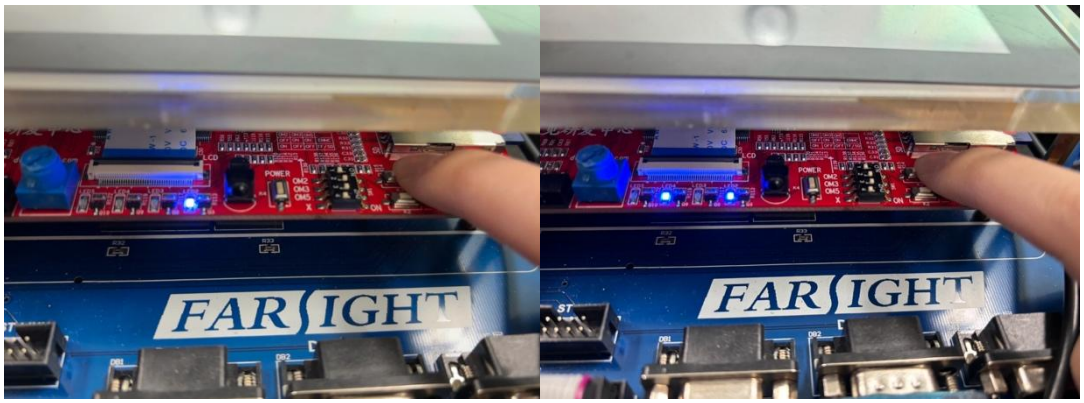
串口中的对应信息：



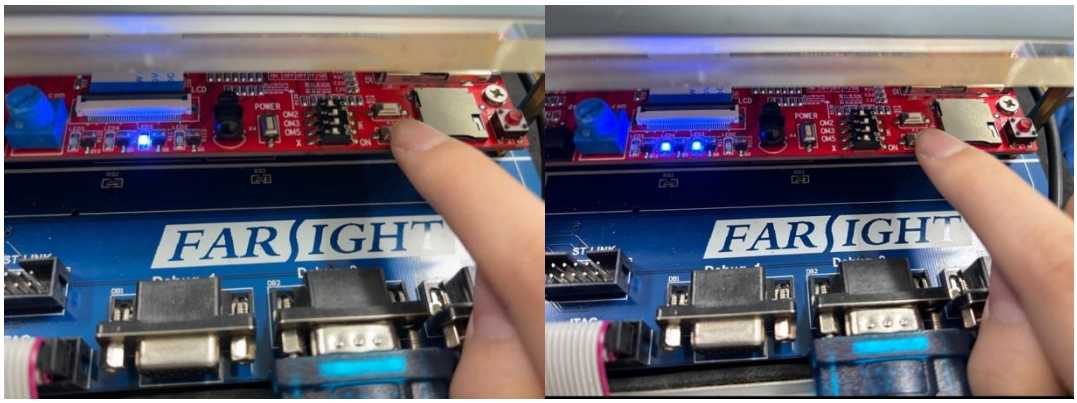


当按下 Key2 或 Key3 的时候观察 LED2 和 LED3 的亮灭情况，通过串口中断也可以看到对应的打印信息。

按下 K3 时，LED2 常量，LED4 闪烁。



按下 K2 时，LED3 常量，LED4 闪烁。



六、实验结论：

本次实验我导入了一个利用 Exynos4412 的 K2、K3 两个 I/O 引脚的中断模式，实现了当按下按键时进入相应的中断处理函数，并处理相应的事件的程序。在程序中，首先对 GPX1_1、GPX1_2 引脚进行设置，使其进入中断模式，并配置中断触发方式。然后在中断处理函数中处理按键事件，并在串口中输出相应的按键信息。通过调试工具的配置和调试，验证了中断处理的正确性。

本次实验使我了解了 ARM 处理器中断的原理和中断处理流程，掌握了中断向量表的设置方法，加深了对 Exynos4412 开发板中 GPIO 引脚中断模式的配置和使用方法的理解。通过调试工具的配置和调试过程，提高了对嵌入式系统中断处理的调试技能。

本次实验使我对 GPIO 中断的应用场景有了更深入的认识。

练习题：

1. 简述 ARM 有几种异常，以及每种异常对应的处理器工作模式。

答：有以下七种。

1) 复位（RESET)：当处理器复位引脚有效时，系统产生复位异常中断。复位异常中断通常在系统加电和系统复位时发生，直接跳转到复位中断向量处执行称为软复位。
2) 未定义的指令（UDEF）：当 ARM 处理器或者是系统中的协处理器认为当前指令未定义时，会产生未定义的指令异常中断。
3) 软件中断（SWI）：这是一个由 SWI 指令触发的中断。
4) 指令预取终止（PrefechAbort)：如果处理器预取的指令的地址不存在，或者该地址不允许当前指令访问，就会产生指令预取终止异常中断。
5) 数据访问终止（DATAABORT）：如果数据访问指令的目标地址不存在，或者该地址不允许当前指令访问就会产生数据访问终止异常中断。
6) 外部中断请求（IRQ）：当处理器的外部中断请求引脚有效，而且 CPSR 的寄存器的 I 控制位被清除时，就会产生外部中断请求异常中断。
7) 快速中断请求（FIQ）：当处理器的外部快速中断请求引脚有效，并且 CPSR 的 F 控制位被清除时，就会产生外部中断请求异常中断。

2. 使用向量中断编写一个平台上其它按键中断的程序，并实现上升沿、下降沿、 高电平、低电平等促发方式。

```
1. #include "stm32f4xx.h"  // 根据您的平台选择合适的头文件
2.
3. // 定义中断触发方式
4. #define RISING_EDGE 1
5. #define FALLING_EDGE 2
6. #define HIGH_LEVEL 3
7. #define LOW_LEVEL 4
8.
9. // 按键引脚和端口定义
10.#define KEY_PIN GPIO_PIN_0  // 假设按键连接到 GPIOA 的第 0 引脚
11.#define KEY_PORT GPIOA
12.
```

```c
void GPIO_Config(void);
void EXTI_Config(uint32_t trigger);
void EXTI0_IRQHandler(void);  // 处理按键中断

int main(void) {
    // 配置 GPIO 和中断
    GPIO_Config();
    EXTI_Config(RISING_EDGE);  // 设置中断触发方式为上升沿

    while (1) {
        // 主循环，执行其他任务
    }
}

// 配置 GPIO 为输入
void GPIO_Config(void) {
    GPIO_InitTypeDef GPIO_InitStruct;

    __HAL_RCC_GPIOA_CLK_ENABLE();  // 使能 GPIOA 时钟

    GPIO_InitStruct.Pin = KEY_PIN;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;  // 初始为上升沿触发
    GPIO_InitStruct.Pull = GPIO_NOPULL;  // 不启用上拉/下拉电阻
    HAL_GPIO_Init(KEY_PORT, &GPIO_InitStruct);

    // 配置 NVIC 中断
    HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0);  // 设置优先级
    HAL_NVIC_EnableIRQ(EXTI0_IRQn);  // 使能中断
}

// 配置中断触发方式
void EXTI_Config(uint32_t trigger) {
    if (trigger == RISING_EDGE) {
        EXTI->IMR |= EXTI_IMR_IM0;  // 使能中断
        EXTI->FTSR &= ~EXTI_FTSR_FT0;  // 禁用下降沿触发
        EXTI->RTSR |= EXTI_RTSR_TR0;  // 启用上升沿触发
    } else if (trigger == FALLING_EDGE) {
        EXTI->IMR |= EXTI_IMR_IM0;
        EXTI->RTSR &= ~EXTI_RTSR_TR0;  // 禁用上升沿触发
        EXTI->FTSR |= EXTI_FTSR_FT0;  // 启用下降沿触发
    } else if (trigger == HIGH_LEVEL) {
        EXTI->IMR |= EXTI_IMR_IM0;
```

```
        EXTI->RTSR |= EXTI_RTSR_TR0;  // 启用上升沿触发
        EXTI->FTSR |= EXTI_FTSR_FT0;  // 启用下降沿触发
    } else if (trigger == LOW_LEVEL) {
        EXTI->IMR |= EXTI_IMR_IM0;
        EXTI->RTSR &= ~EXTI_RTSR_TR0;  // 禁用上升沿触发
        EXTI->FTSR &= ~EXTI_FTSR_FT0;  // 禁用下降沿触发
    }
}

// 按键中断服务程序
void EXTI0_IRQHandler(void) {
    if (__HAL_GPIO_EXTI_GET_IT(KEY_PIN) != RESET) {
        __HAL_GPIO_EXTI_CLEAR_IT(KEY_PIN);  // 清除中断标志

        // 根据需求处理按键事件
        // 例如：按键被按下时，切换 LED 状态等
    }
}
```

1. GPIO 配置段：

该代码配置了按键连接的 GPIO 引脚为输入模式，并使能其外部中断功能。首先，通过 __HAL_RCC_GPIOA_CLK_ENABLE() 启用 GPIOA 端口的时钟，然后将按键引脚设置为“上升沿触发中断模式”。HAL_GPIO_Init(KEY_PORT, &GPIO_InitStruct) 将配置应用到指定的端口和引脚。最后，通过 HAL_NVIC_SetPriority 和 HAL_NVIC_EnableIRQ 设置并使能 NVIC 中断，确保按键按下时会触发中断。

2. 中断触发方式配置段：

该代码根据不同的触发方式（上升沿、下降沿、高电平或低电平），配置了外部中断控制器（EXTI）的行为。通过设置 EXTI->RTSR 和 EXTI->FTSR 寄存器来选择适当的触发模式。例如，如果选择上升沿触发，它会启用 RTSR 的相应位，并禁用 FTSR 的位，从而使得按键的上升沿能够触发中断。不同的配置方式决定了系统如何响应按键输入。

七、指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

八、备注：

注：报告内的项目或内容设置，根据实际情况可加以调整和补充，写不下可另附一页。